

UNIVERSITÉ / ÉCOLE SUPÉRIEURE

D'INGÉNIERIE

Département Informatique & Systèmes d'Information



PULSEGRID

Unified Intelligence. Zero Overhead.

Rapport de Projet de Fin d'Études (PFE)

Plateforme SaaS de Business Intelligence Multi-Entreprises

Présenté par :

Jawher [NOM]

Filière : Génie Logiciel

Encadrant(e) :

[Titre Prénom NOM]

Année universitaire : 2024 – 2025

*À mes parents, pour leur soutien indéfectible
et leur confiance en chacune de mes ambitions.*

*À mes enseignants, pour la rigueur et la passion
qu'ils ont su transmettre tout au long de ma formation.*

*À tous ceux qui croient qu'un bon logiciel
commence par une vraie compréhension du problème.*

Remerciements

Je tiens à exprimer ma sincère gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce projet de fin d'études.

En premier lieu, je remercie chaleureusement mon encadrant(e), [Titre Prénom NOM], pour sa disponibilité, ses conseils avisés, et la pertinence de ses orientations tout au long de ce projet. Sa rigueur scientifique et son exigence bienveillante ont été une source de motivation constante.

Je remercie également l'équipe de **UNILOG Tunisie** pour m'avoir accueilli et m'avoir permis de travailler dans un environnement stimulant et innovant. Leurs conseils pratiques et leur expérience du domaine des ERP ont été particulièrement enrichissants.

Je remercie également l'ensemble du corps enseignant du département Informatique pour la qualité de la formation dispensée, qui m'a fourni les fondements théoriques et pratiques nécessaires à la conception de ce système.

Mes remerciements s'adressent aussi à mes camarades de promotion pour les échanges intellectuels enrichissants et le soutien mutuel tout au long de

ce parcours académique.

Enfin, je remercie ma famille pour son soutien moral et sa patience, qui ont été essentiels à l'accomplissement de ce travail.

Résumé

Ce projet de fin d'études présente la conception et le développement de **PulseGrid**, une plateforme **SaaS** (Software as a Service) de **Business Intelligence** (Informatique Décisionnelle) innovante. Dans un contexte où les dirigeants gèrent simultanément plusieurs activités commerciales, chacune disposant de ses propres tableaux de bord, outils de reporting et interfaces de connexion, la fragmentation de l'information constitue un frein majeur à la prise de décision efficace.

PulseGrid permet aux opérateurs multi-entreprises de centraliser leurs indicateurs clés (**KPI** - Key Performance Indicators) depuis diverses API REST (Stripe, Shopify, etc.) dans un tableau de bord unifié. L'originalité du projet réside dans son architecture « **Zero Storage** » (Stockage Zéro) : aucune donnée métier n'est stockée sur nos serveurs. Le chiffrement **client-side** (côté client) via l'API **Web Crypto** (AES-GCM 256 bits) garantit une confidentialité totale. L'intégration d'un moteur d'IA basé sur **Llama 3.3 (via OpenRouter)** permet d'automatiser l'analyse des données et de générer des rapports intelligents en temps réel. La solution est développée avec **React**, **Vite** et **Tailwind CSS**.

Mots-clés : Business Intelligence, SaaS, React, Vite, Cryptographie, Intelligence Artificielle, Zero Storage, Tableau de bord, KPI.

Abstract

This report presents the design and development of PulseGrid, a SaaS Business Intelligence platform targeting multi-business operators. Information fragmentation across multiple ventures severely hampers effective decision-making.

PulseGrid centralizes all Key Performance Indicators into a single workspace. No data storage backend is required — data is fetched in real time from third-party REST APIs. Credentials are protected through AES-GCM 256-bit encryption via the browser’s Web Crypto API.

The platform features an AI analysis engine leveraging Llama 3.3 via the OpenRouter API for automated metric analysis, anomaly detection, and natural-language summaries. The frontend is built with React, Vite, Tailwind CSS, and ApexCharts, following solid principles and best practices.

Keywords : Business Intelligence, SaaS, React, Vite, REST API, Web Crypto, OpenRouter, LLM, Dashboard, KPI.

Table des matières

Remerciements	2
Résumé	4
Introduction	1
1 Étude préalable	4
1.1 Recueil	4
1.1.1 Mission et Objectifs	5
1.1.2 Analyse de l'existant	6
1.2 Conclusion	13
2 Modélisation des besoins	15
2.1 Objectif du site	15
2.2 Public visé	16

2.3	Services offerts	17
2.3.1	Diagramme de cas d'utilisation	18
2.3.2	Identification et représentation des scénarios	21
2.3.3	Diagrammes de séquences	25
2.4	Contenu informationnel	29
2.5	Conclusion	30
3	Modélisation conceptuelle	31
3.1	Architecture statique de PulseGrid	31
3.1.1	Architecture du Back Office	32
3.1.2	Architecture du Front Office	33
3.2	Diagramme d'états transitions	34
3.3	Aspect visuel	35
3.4	Conclusion	37
4	Réalisation du site web	38
4.1	Environnement de réalisation	38

4.1.1	Environnement matériel	39
4.1.2	Environnement logiciel	39
4.2	Vers la réalisation du site web	41
4.2.1	Passage du diagramme de classes au modèle logique de données	41
4.2.2	Modélisation physique	43
4.3	Schéma de navigation du site de PulseGrid	44
4.4	Réalisation de l'aspect visuel du site de PulseGrid	45
4.5	Conclusion	49
Conclusion Générale		51
Bibliographie		53
Webliographie		54
Glossaire		55
Annexes		58

Table des figures

2.1	Diagramme des cas d'utilisation global de la plateforme PulseGrid	20
2.2	Diagramme de séquence pour l'authentification et la dérivation de clé	26
2.3	Diagramme de séquence pour le rafraîchissement des données d'un widget	27
2.4	Flux de traitement et d'anonymisation vers l'intelligence artificielle	28
3.1	Diagramme de classes simplifié des entités du domaine . . .	34
3.2	Croquis initial et wireframe de la structure du Dashboard . .	36
4.1	Écosystème technologique principal	41
4.2	Interface finale du Dashboard PulseGrid avec widgets actifs .	46

4.3	Panneau latéral (Drawer) de configuration d'un nouveau widget REST	46
4.4	Panneau d'analyse IA affichant un Project Brief généré . . .	47
4.5	Version mobile optimisée de PulseGrid	48
4.6	Interface principale en cours d'utilisation (Capture d'écran de production)	49

Liste des tableaux

1.1	Impact multidimensionnel de la fragmentation des données sur la gestion d'entreprise	7
1.2	Fiche signalétique de l'entreprise UNILOG Tunisie	8
1.3	Catalogue des modules de la suite ERP UniGes par UNILOG	10
1.4	Tableau comparatif des solutions de Business Intelligence du marché	12
2.1	Identification des acteurs du système PulseGrid	19
2.2	Description textuelle du cas d'utilisation CU-01 : Création de projet	22
2.3	Description textuelle du cas d'utilisation CU-02 : Ajout d'un widget	23
2.4	Description textuelle du cas d'utilisation CU-03 : Analyse IA	24

4.1	Tableau de l'environnement logiciel et de l'outillage de développement	40
4.2	Rapport de l'audit de sécurité minimaliste	61

Introduction

L’essor de l’entrepreneuriat numérique et la multiplication des modèles économiques basés sur les données ont engendré une nouvelle réalité pour les opérateurs multi-entreprises : la nécessité de piloter simultanément plusieurs activités commerciales, chacune générant des flux de données hétérogènes, stockés dans des outils distincts et accessibles via des interfaces différentes.

Cette fragmentation de l’information crée un coût opérationnel invisible mais significatif. Selon plusieurs études sur la productivité des dirigeants, la navigation entre différents outils de reporting peut absorber entre 45 minutes et 2 heures par jour — un temps précieux soustrait à la prise de décision stratégique.

Par ailleurs, l’avènement des modèles de langage de grande taille (LLM) et des API d’inférence ultra-rapides comme OpenRouter ouvre une nouvelle perspective : celle d’un assistant analytique intégré, capable d’interpréter des données métier complexes et de les restituer en langage naturel, en quelques secondes.

PulseGrid naît d’un besoin concret et vécu : un opérateur gérant plu-

sieurs entreprises simultanément — boutique e-commerce, application SaaS, portefeuille immobilier, agence de marketing — se retrouve chaque matin à enchaîner les connexions, les chargements de pages, les exports de rapports, sans jamais avoir une vision globale cohérente.

L'idée centrale est radicale dans sa simplicité : connecter n'importe quelle API REST, configurer un widget en quelques clics, et disposer instantanément d'un tableau de bord intelligent, unifié et analysé par une IA. Sans base de données propre, sans entrepôt de données, sans infrastructure lourde.

Ce projet de fin d'études vise à concevoir et développer PulseGrid, une plateforme SaaS de Business Intelligence répondant aux objectifs suivants :

1. Centraliser les KPI de plusieurs entreprises dans un espace de travail unique et unifié.
2. Permettre la connexion à n'importe quelle API REST via un système de widgets configurables.
3. Garantir la sécurité des identifiants API par un chiffrement client-side AES-GCM via Web Crypto.
4. Intégrer une analyse IA en temps quasi-réel via le modèle Llama 3.3 (OpenRouter).
5. Offrir une expérience utilisateur premium et responsive via React et Tailwind CSS.
6. Respecter une architecture logicielle propre, maintenable et scalable.

Ce rapport détaille le cycle complet de réalisation du projet PulseGrid, structuré en quatre chapitres :

— **Chapitre 1 : Étude préalable.** Ce chapitre est consacré au recueil

des informations, avec la définition de la mission et des objectifs du projet. Il comprend également une analyse détaillée de l'existant, incluant la présentation de l'entreprise d'accueil UNILOG et une étude comparative des sites web similaires sur le marché.

- **Chapitre 2 : Modélisation des besoins.** Nous y définirons l'objectif global du site, le public visé, et les services offerts. Cette section s'appuiera sur des diagrammes UML, notamment le diagramme de cas d'utilisation, l'identification des scénarios, ainsi que les diagrammes de séquences pour les interactions clés, pour enfin détailler le contenu informationnel.
- **Chapitre 3 : Modélisation conceptuelle.** Ce chapitre présente l'architecture statique de PulseGrid, en distinguant l'architecture du Back Office et celle du Front Office. Nous y aborderons également le diagramme d'états transitions et une présentation détaillée de l'aspect visuel (maquettes et conception de l'interface utilisateur).
- **Chapitre 4 : Réalisation du site web.** Le dernier chapitre se concentre sur l'implémentation. Il décrit l'environnement matériel et logiciel, puis explique le passage du diagramme de classes au modèle logique, et enfin la modélisation physique. Nous concluons par le schéma de navigation et des captures de la réalisation finale de l'aspect visuel de PulseGrid.

Chapitre 1

Étude préalable

Ce premier chapitre situe le projet dans son environnement professionnel et académique, en présentant le contexte général, l'organisme d'accueil et en analysant les enjeux métiers auxquels répond PulseGrid. Il permet de poser les fondations nécessaires à la bonne compréhension des objectifs du projet et des solutions qui seront proposées dans les chapitres suivants.

1.1 Recueil

La phase de recueil d'informations est une étape primordiale dans tout processus d'ingénierie logicielle. Elle permet de circonscrire le problème à résoudre, de définir clairement les objectifs à atteindre et de comprendre l'environnement dans lequel la solution va s'insérer. Cette section est consacrée à la définition de la mission de PulseGrid et à l'analyse de l'existant.

1.1.1 Mission et Objectifs

PulseGrid s'inscrit dans le cadre d'un Projet de Fin d'Études en Génie Logiciel, orienté vers la conception et le développement d'une solution SaaS complète, de la spécification des besoins jusqu'au déploiement. Il mobilise des compétences en architecture logicielle, développement frontend moderne, sécurité applicative et intégration d'intelligence artificielle.

La mission principale de ce projet est de fournir une application web de type SaaS qui s'adresse à un marché ciblé : les opérateurs d'entreprises multiples, les agences gérant plusieurs clients, les holdings et les indépendants disposant de plusieurs sources de revenus.

Les objectifs spécifiques sont les suivants :

- **Unification des données** : Fournir un point d'accès unique pour l'ensemble des indicateurs de performance (KPI) des différentes activités commerciales d'un utilisateur.
- **Interopérabilité universelle** : Permettre l'intégration de toute source de données exposant une API REST, qu'il s'agisse de plateformes e-commerce (Shopify, WooCommerce), de passerelles de paiement (Stripe, PayPal) ou de systèmes d'analyse (Google Analytics).
- **Sécurité absolue** : Garantir que les identifiants et clés API des utilisateurs ne soient jamais compromis ou stockés en clair, répondant

ainsi à la préoccupation majeure de la confidentialité des données commerciales.

- **Assistance intelligente** : Doter la plateforme d’une intelligence artificielle capable de synthétiser, d’analyser et de mettre en évidence des anomalies ou des corrélations que l’utilisateur pourrait manquer en raison du volume d’informations.

1.1.2 Analyse de l’existant

La problématique centrale peut se formuler ainsi : *Comment permettre à un opérateur gérant plusieurs entités commerciales hétérogènes d’avoir une vision consolidée, en temps réel, de l’ensemble de ses indicateurs de performance, sans investissement en infrastructure de données ?*

Une journée type d’un opérateur multi-entreprises ressemble à la séquence suivante : connexion au tableau de bord Shopify de la boutique A, export du rapport de ventes, connexion au CRM de l’entreprise B, vérification des métriques Google Analytics, ouverture de Stripe pour contrôler les revenus SaaS, puis répétition du cycle pour chaque entité supplémentaire.

Le coût caché de cette fragmentation est illustré par l’impact multidimensionnel sur la gestion d’entreprise :

Problème	Impact mesurable
Interfaces hétérogènes	Charge cognitive élevée, fatigue et erreurs d'interprétation et de saisie.
Multiples connexions quotidiennes	Perte de temps opérationnel estimée à 45 à 90 minutes perdues par jour en moyenne.
Absence de vue consolidée	Décisions prises sans vision globale cohérente.
Pas d'analyse croisée	Opportunités et corrélations invisibles (ex : AdSpend vs Revenue non mis en regard).
Pas d'alerte intelligente	Anomalies ou baisses soudaines détectées trop tard, avec des conséquences financières (ex : taux de conversion Stripe en chute).
Confidentialité	Risque critique lié au stockage des clés API "Master" sur des serveurs tiers.

TABLE 1.1 – Impact multidimensionnel de la fragmentation des données sur la gestion d'entreprise

Présentation de l'entreprise

J'ai effectué mon stage de projet de fin d'études au sein de **UNILOG Tunisie**, une entreprise de référence basée à Sfax, spécialisée dans l'édition et l'intégration de logiciels de gestion (ERP).

Fiche signalétique :

Raison sociale	UNILOG Tunisie
Produit principal	ERP UniGes
Date de création	2008
Secteur d'activité	Édition et intégration de logiciels de gestion
Adresse	Sfax, Tunisie
Téléphone	+216 70 221 797
Email	contact@unilog.tn
Site web	www.unilog.tn
Marchés couverts	Tunisie, Algérie, Afrique, Europe

TABLE 1.2 – Fiche signalétique de l'entreprise UNILOG Tunisie

Historique :

Fondée en 2008, UNILOG Tunisie est une entreprise spécialisée dans le développement et l'implémentation de solutions de gestion d'entreprise. Depuis sa création, elle a concentré ses efforts sur la conception de l'ERP UniGes, une solution modulaire adaptée aux besoins des entreprises industrielles,

commerciales et de services. Au fil des années, l'entreprise a élargi son offre pour couvrir plusieurs secteurs d'activité et s'est progressivement positionnée comme un acteur de référence dans le domaine des ERP en Tunisie et à l'international.

Missions et valeurs :

UNILOG a pour mission d'accompagner les entreprises dans la structuration et l'optimisation de leurs processus de gestion à travers des solutions logicielles fiables et adaptables. L'entreprise s'appuie sur plusieurs valeurs fondamentales :

- Proximité client et réactivité du support après-vente.
- Qualité et fiabilité des solutions proposées.
- Adaptabilité aux besoins spécifiques de chaque secteur métier.
- Engagement continu dans l'innovation et l'amélioration continue des produits.

Activités et solutions :

UNILOG développe et commercialise l'ERP UniGes, une suite logicielle exhaustive et évolutive. L'offre est composée de plusieurs modules couvrant l'ensemble des fonctions d'une entreprise moderne :

Module	Description
ERP / Comptabilité & Finance	Gestion financière, budgétaire et comptable intégrée.
GPAO	Gestion de la production assistée par ordinateur pour les flux industriels.
GMAO	Gestion de la maintenance assistée par ordinateur des équipements.
GRH	Gestion des ressources humaines, pointage et traitement de la paie.
CRM	Gestion de la relation client, des devis aux contrats.
Business Intelligence	Tableaux de bord, reporting dynamique et outils d'aide à la décision.
E-commerce	Solution de commerce électronique nativement intégrée à l'ERP.
Management Qualité	Suivi qualité et amélioration continue (normes ISO, traçabilité).
Gestion Chantier BTP	Suivi de projets, coûts et ressources dans le secteur de la construction.

TABLE 1.3 – Catalogue des modules de la suite ERP UniGes par UNILOG

Ces solutions couvrent des secteurs variés et exigeants, tels que l'industrie pharmaceutique, l'agroalimentaire, la mécanique de précision, le BTP, la grande distribution, l'expertise comptable, et le décolletage.

Structure et organisation :

L'entreprise est organisée autour de plusieurs pôles d'expertise complémentaires qui collaborent étroitement :

- **Le département de développement logiciel** : En charge de la conception, de l'architecture et de l'évolution technique de tous les modules de la suite UniGes. C'est au sein de ce département que j'ai été intégré.
- **Le département consulting et intégration** : Chargé du déploiement des solutions chez les clients, du paramétrage spécifique et de la formation des utilisateurs finaux.
- **Le département support et après-vente** : Assurant la continuité opérationnelle, la maintenance évolutive et la résolution rapide des incidents clients.
- **Le département commercial et marketing** : Gérant la relation avec les prospects, l'expansion sur de nouveaux marchés et la gestion des partenaires.

Au sein du département Développement, j'ai participé activement à l'innovation sur les axes de Business Intelligence, en apportant des concepts modernes comme le frontend React, les architectures sans état (stateless) et

l'intégration de modèles d'Intelligence Artificielle.

Étude de sites web similaires

Les outils BI existants comme Tableau, Power BI ou Looker répondent aux besoins des grandes entreprises mais imposent des contraintes rédhibitoires pour les profils ciblés par PulseGrid. Ces systèmes requièrent la mise en place de bases de données, d'entrepôts de données complexes (Data Warehouses) ou l'installation de connecteurs propriétaires.

Avant d'entamer le développement, une analyse comparative (benchmark) détaillée a été conduite pour délimiter clairement la valeur ajoutée de PulseGrid par rapport aux géants du secteur.

Critère	Tableau	Power BI	Metabase	Databox	
Type de solution	Enterprise	Enterprise	Open Source	BI SaaS	BI
Zéro Backend	Non	Non	Non	Partiel	
IA Intégrée	Limité	Partiel	Non	Non	Oui (G
Support REST Universel	Non	Non	Oui	Non	
Coût et Complexité	Très élevé	Moyen	Auto-hébergé	Élevé	Acce
Sécurité des clés (Local)	Faible	Moyenne	Moyenne	Faible	Absol

TABLE 1.4 – Tableau comparatif des solutions de Business Intelligence du marché

Le benchmarking révèle qu'aucune solution n'intègre nativement d'IA

général pour l'extraction automatique de métriques tout en garantissant un "stockage zéro" côté serveur.

- **Tableau & Power BI** : Solutions extrêmement puissantes pour la manipulation de grands volumes de données historiques, mais elles nécessitent que l'entreprise dispose d'un pipeline d'extraction de données (ETL) et d'un entrepôt centralisé.
- **Metabase** : Outil excellent pour interroger directement une base SQL existante, mais inadapté pour un entrepreneur souhaitant afficher des données depuis 5 API SaaS différentes sans avoir à coder lui-même un script d'importation vers une base SQL.
- **Databox** : Le concurrent le plus proche en termes d'usage. Il propose des tableaux de bord pré-construits. Cependant, il limite l'utilisateur à ses intégrations officielles et stocke les données récupérées sur ses propres serveurs.

1.2 Conclusion

Ce premier chapitre a permis de définir le contexte général dans lequel évolue le projet PulseGrid. L'étude de l'existant a mis en exergue une lacune importante sur le marché des outils de Business Intelligence : la gestion complexe des API hétérogènes pour les petites et moyennes structures, avec

les risques de confidentialité inhérents. En présentant l'organisme d'accueil, UNILOG, et en analysant les solutions concurrentes, nous avons justifié la pertinence de l'approche « Zero Storage » et « API-first ». Ces éléments fondamentaux guideront la conception et la modélisation des besoins dans le chapitre suivant.

Chapitre 2

Modélisation des besoins

La modélisation des besoins est une étape charnière du cycle de développement logiciel. Elle permet de traduire les exigences abstraites ou générales en spécifications formelles compréhensibles tant par les acteurs fonctionnels que par les équipes techniques. Dans ce chapitre, nous définirons l'objectif global du système, analyserons le public ciblé et décrirons les services offerts via des diagrammes de cas d'utilisation et de séquences.

2.1 Objectif du site

L'objectif fondamental de PulseGrid est de concevoir et développer une plateforme SaaS qui permet à un opérateur gérant plusieurs entités commerciales d'avoir une vision consolidée, en temps réel, de l'ensemble de ses indicateurs de performance, sans investissement en infrastructure de données.

PulseGrid propose une approche diamétralement opposée aux méthodes classiques : au lieu d’ingérer, de traiter et de stocker des téraoctets de données métier, la plateforme agit comme une *grille intelligente*. Elle récupère les données à la demande depuis les API REST des outils métier existants (boutiques en ligne, services de paiement, outils d’analytique), les normalise à la volée directement dans le navigateur, et les présente dans un tableau de bord unifié, configurable par l’utilisateur via un système de widgets flexibles.

2.2 Public visé

L’analyse du marché et la définition des *personas* ont permis de délimiter avec précision le public visé. Les acteurs cibles sont principalement des professionnels pour qui le temps est une ressource critique et l’information fragmentée un risque stratégique.

- **Entrepreneurs multi-activités** : Des dirigeants qui possèdent simultanément une boutique de commerce électronique, une agence de services et peut-être des investissements immobiliers locatifs, générant des données de différentes origines (Shopify, Stripe, logiciels comptables).
- **Agences de marketing digital** : Des structures gérant des campagnes pour des dizaines de clients différents, ayant besoin de visualiser

rapidement les budgets publicitaires dépensés sur Meta ou Google Ads face aux conversions enregistrées.

- **Fondateurs de SaaS et startups** : Des créateurs de logiciels en ligne ayant besoin de suivre les indicateurs de croissance (MRR, Churn rate, taux de conversion) en connectant directement leur base de données ou des services tiers sans avoir à construire leur propre interface d'administration complexe.

Ce public est technophile mais ne dispose pas nécessairement des compétences en ingénierie de données ni du budget pour déployer un système de Business Intelligence complexe d'entreprise.

2.3 Services offerts

Afin de répondre aux besoins de ce public, PulseGrid offre une suite de services articulés autour de plusieurs modules clés :

1. **Module Authentification et Comptes** : Inscription, connexion sécurisée, gestion du profil et récupération de mot de passe.
2. **Module Gestion des Projets** : Création et gestion de différents environnements de tableaux de bord (un par entreprise ou par domaine d'analyse).
3. **Module Moteur de Widgets** : Cœur du système permettant d'ajou-

ter, configurer et positionner des visualisations graphiques connectées à des sources de données externes (API REST) en temps réel.

4. **Module Intelligence Artificielle** : Intégration avancée avec les modèles LLM (Llama 3.3) pour générer des rapports d'analyse, identifier des tendances et repérer d'éventuelles anomalies parmi l'ensemble des données des widgets.

2.3.1 Diagramme de cas d'utilisation

Pour modéliser l'interaction entre les utilisateurs et le système PulseGrid, nous avons identifié plusieurs acteurs.

Acteur	Type	Rôle Principal
Opérateur (Utilisateur)	Acteur humain	Crée les projets, configure les widgets, connecte ses API tierces et consulte les analyses IA.
API Tierce	Acteur externe	Fournit les données brutes sous format JSON en réponse aux requêtes (Stripe, Shopify, etc.).
OpenRouter (IA)	Acteur externe	Agrégateur de modèles d'Intelligence Artificielle fournissant les analyses génératives.
Administrateur	Acteur humain	Gère la maintenance technique de la plateforme, les quotas et le support utilisateur.

TABLE 2.1 – Identification des acteurs du système PulseGrid

Le diagramme suivant illustre les cas d'utilisation globaux du système PulseGrid pour l'acteur principal (l'utilisateur).

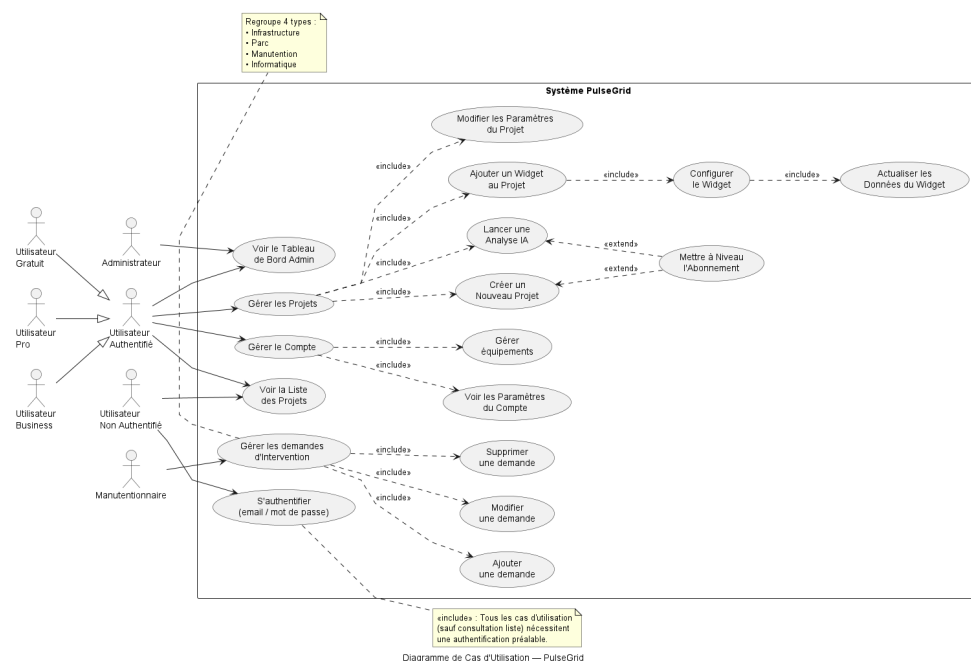


FIGURE 2.1 – Diagramme des cas d'utilisation global de la plateforme PulseGrid

2.3.2 Identification et représentation des scénarios

Afin de mieux comprendre le comportement attendu du système face aux actions de l'utilisateur, nous décrivons ici les scénarios textuels pour trois des cas d'utilisation les plus importants de l'application.

Champ
Nom
Acteur principal
Précondition

Champ
Nom
Acteur principal
Précondition

Champ
Nom
Acteur principal
Précondition

2.3.3 Diagrammes de séquences

Les diagrammes de séquences précisent l'aspect dynamique du système en modélisant l'ordre chronologique des messages échangés entre les différents acteurs et objets du système lors de l'exécution des cas d'utilisation.

Séquence d'authentification et chiffrement

Ce premier diagramme illustre le processus d'authentification et l'étape critique de dérivation de clé. Afin de garantir le modèle "Zero Storage", le mot de passe de l'utilisateur n'est pas uniquement utilisé pour l'authentification Firebase, il sert également à générer une clé de chiffrement locale (via PBKDF2) qui sera conservée exclusivement en mémoire vive dans le navigateur.

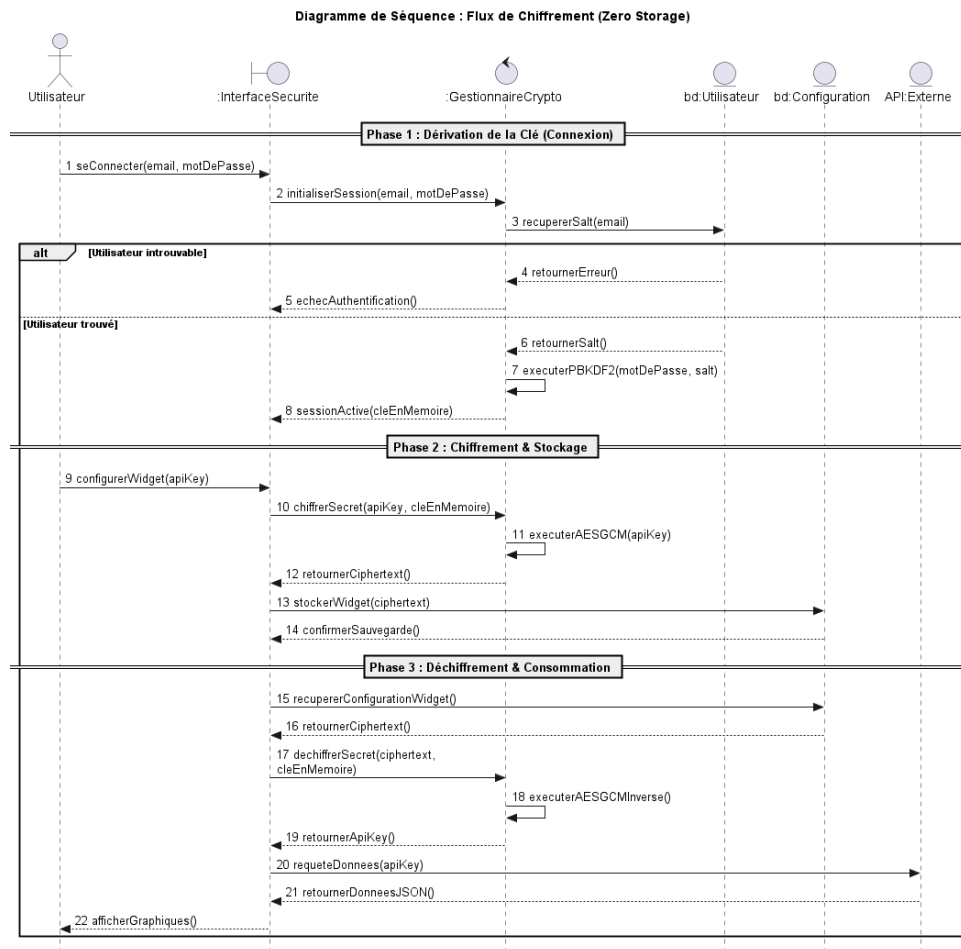


FIGURE 2.2 – Diagramme de séquence pour l’authentification et la dérivation de clé

L’initialisation de ce cycle est ce qui permet par la suite d’interagir avec la base de données en s’assurant que tout secret API envoyé est d’abord rendu illisible.

Séquence de récupération des données des widgets

Une fois l'utilisateur authentifié et les widgets configurés, le tableau de bord doit afficher les données. Ce diagramme montre comment le système récupère la configuration chiffrée, la déchiffre localement à l'aide de la clé en mémoire, puis effectue la requête *directement* auprès de l'API tierce, évitant ainsi le passage par un serveur backend intermédiaire.

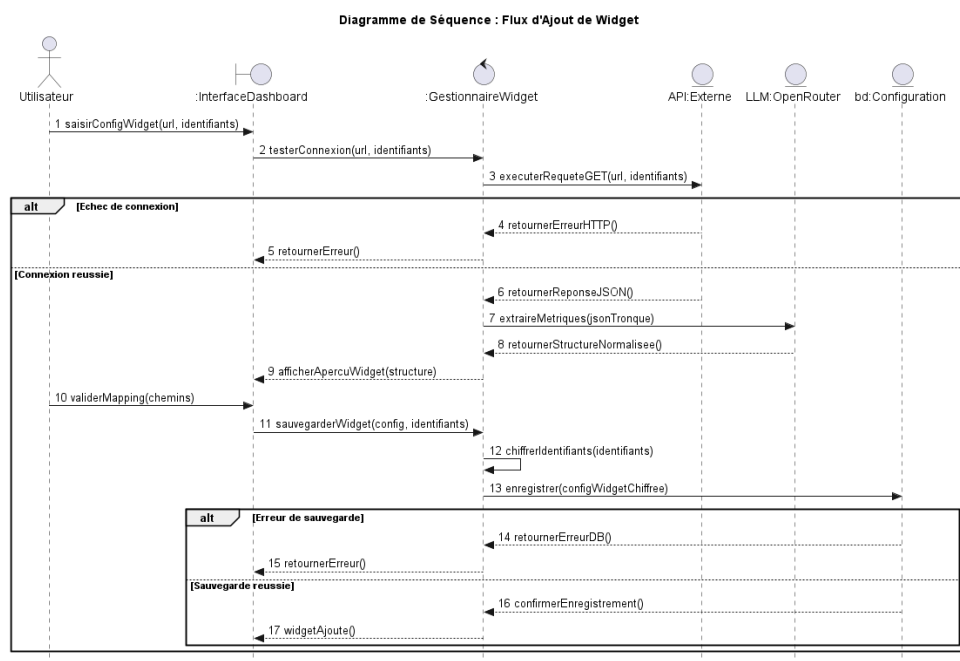


FIGURE 2.3 – Diagramme de séquence pour le rafraîchissement des données d'un widget

Séquence d'analyse Intelligence Artificielle

Ce dernier diagramme expose la séquence déclenchée lorsque l'utilisateur sollicite une analyse de ses KPI. Pour des raisons de confidentialité, les réponses brutes des API tierces ne sont pas envoyées à l'IA. Le système extrait uniquement les indicateurs, les normalise, puis les achemine vers un proxy (backend Firebase) qui se charge de dialoguer avec OpenRouter (Llama 3.3).

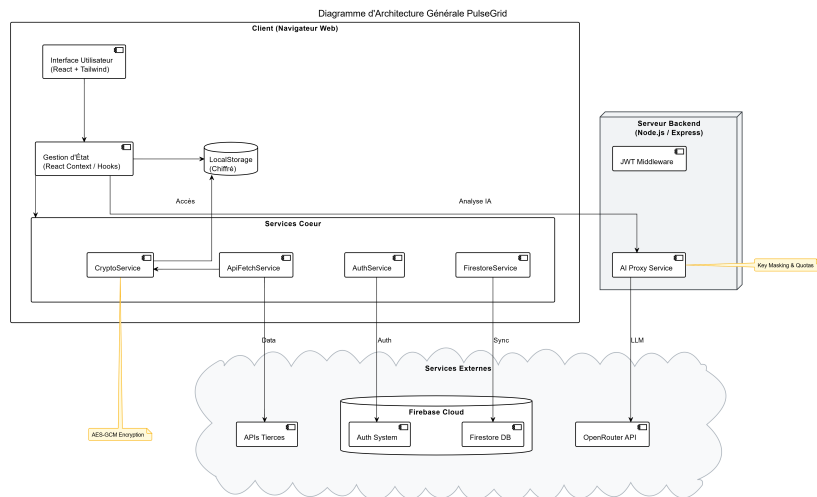


FIGURE 2.4 – Flux de traitement et d'anonymisation vers l'intelligence artificielle

Note : Le diagramme ci-dessus illustre de manière architecturale le cheminement de la donnée vers l'IA, soulignant la séparation stricte entre le flux direct (données métiers) et le flux indirect (analyse agrégée via Proxy).

2.4 Contenu informationnel

Pour soutenir l'ensemble de ces services et scénarios, le système repose sur un ensemble d'exigences non fonctionnelles strictes, qui dictent la manière dont le contenu et l'architecture doivent être manipulés :

- **Performance et Réactivité** : Le chargement d'un tableau de bord avec une dizaine de widgets doit s'effectuer de manière asynchrone (toutes les requêtes API sont lancées en parallèle). Le rendu doit se faire en moins de 1.5 seconde.
- **Confidentialité Cryptographique** : Toute configuration de widget contenant une clé (Secret API, Token) doit obligatoirement être chiffrée en AES-GCM 256 bits avant son stockage dans la base Firestore.
- **Résilience et Gestion des Erreurs** : Si l'API tierce de l'un des widgets ne répond pas (Timeout) ou renvoie une erreur 401 (Non autorisé), ce widget précis doit afficher un état d'erreur isolé sans bloquer ni l'interface ni le chargement des autres graphiques.

Le contenu manipulé s'organise autour d'entités logiques simples : L'**Utilisateur** possède des **Projets**. Chaque projet contient des **Widgets**. Chaque widget possède une définition de visualisation et un **Endpoint REST**.

2.5 Conclusion

Dans ce chapitre, nous avons défini l’objectif principal et la cible de la solution PulseGrid, tout en modélisant les besoins via des diagrammes de cas d’utilisation et de séquences. Ces diagrammes ont mis en lumière la particularité et l’exigence du système : un flux asynchrone sécurisé, où la donnée transite différemment selon qu’elle est affichée à l’écran ou envoyée pour analyse sémantique. Ces spécifications fonctionnelles nous permettent maintenant de concevoir l’architecture détaillée de l’application dans le prochain chapitre.

Chapitre 3

Modélisation conceptuelle

La modélisation conceptuelle constitue l'étape de transition entre la spécification des besoins et la réalisation technique. Elle s'attache à définir la structure interne du système, à travers la description de son architecture statique, des états de ses entités métiers, et enfin, par une première projection de son aspect visuel.

3.1 Architecture statique de PulseGrid

L'architecture statique de PulseGrid repose sur un paradigme de découplage fort, structuré par la séparation stricte entre le flux d'authentification et le flux de données métier. Contrairement aux solutions BI classiques, PulseGrid ne sert que de « **passe-plat** » (**Proxy**) pour les configurations chiffrées, tandis que les données réelles circulent directement entre le navigateur de l'utilisateur et les API tierces. Ce paradigme est appelé « **Zero**

Storage » (Zéro Stockage).

3.1.1 Architecture du Back Office

Bien que la plateforme minimise la charge côté serveur pour le traitement des données métiers, un *Back Office* (Backend) minimaliste est indispensable pour la gouvernance du système.

Ce backend, hébergé sur l'infrastructure **Firebase** (Node.js/Express via Cloud Functions), remplit des rôles très spécifiques :

- **Gestion des identités** : Création des comptes, gestion des sessions sécurisées (JWT) et réinitialisation des accès via Firebase Auth.
- **Stockage de la configuration** : Base de données NoSQL (Firestore) conservant l'arborescence des projets et des widgets. La particularité ici réside dans le fait que les champs sensibles (clés API) stockés dans cette base ne sont que des suites de caractères chiffrés (*Ciphertext* en Base64), totalement illisibles pour les administrateurs du backend.
- **Proxy Intelligence Artificielle** : Pour éviter que la clé API "Master" d'OpenRouter ne soit exposée publiquement dans le code côté client, le backend fait office de Proxy sécurisé. Il reçoit le condensé anonymisé des métriques, valide le quota de l'utilisateur, puis interroge le modèle Llama 3.3.

3.1.2 Architecture du Front Office

Le *Front Office* (Frontend) représente le véritable "moteur" de l'application. Développé en **React** et propulsé par **Vite**, il est responsable d'exécuter l'intégralité de la logique de récupération, de déchiffrement et d'affichage.

- **Interface Utilisateur (UI)** : Construit de manière déclarative en composants réutilisables, stylisés avec Tailwind CSS pour garantir un design fluide et *responsive*.
- **Moteur de Cryptographie** : Un module intégré utilisant nativement la *Web Crypto API* du navigateur pour dériver les clés et déchiffrer les secrets de configuration en mémoire vive, isolant ainsi la cryptographie des serveurs.
- **Gestionnaire de Requêtes (Widget Engine)** : Un système asynchrone chargé de lire la configuration de chaque widget, de construire la requête HTTP correspondante, d'y injecter la clé API déchiffrée, et d'extraire la valeur JSON ciblée via une mécanique de *Data Mapping*.

Pour structurer ces entités au sein du code source, un diagramme de classes schématise le modèle de domaine du Front Office :

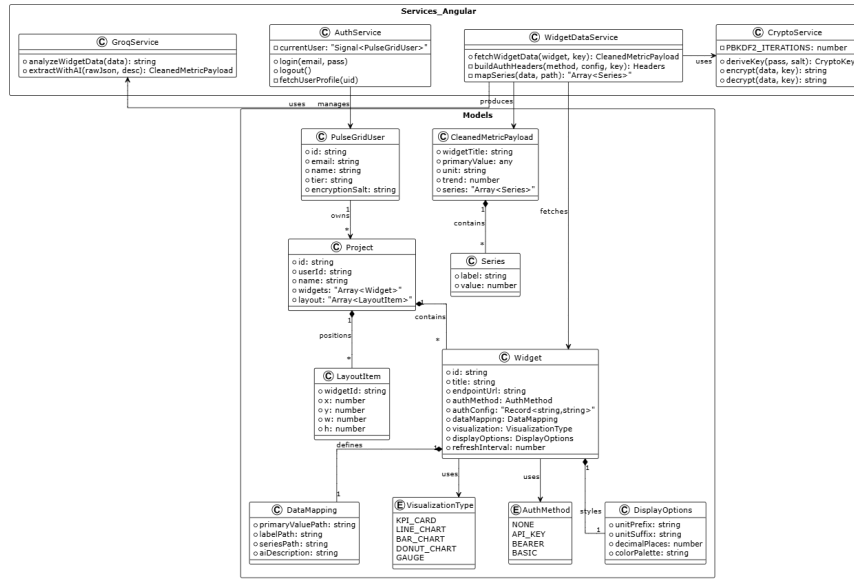


FIGURE 3.1 – Diagramme de classes simplifié des entités du domaine

3.2 Diagramme d'états transitions

Le cycle de vie de la configuration cryptographique est l'un des aspects les plus délicats de l'architecture. Le système traverse différents états pour s'assurer que les clés en mémoire ne soient jamais exposées au-delà de la session active.

Le diagramme d'états transitions du flux cryptographique peut être décrit de la façon suivante :

1. **État Initial (Déconnecté)** : La session est vierge. Seul le *Salt* (sel) est stocké de manière persistante.
2. **État de Transition (Authentification)** : L'utilisateur saisit son mot de passe. Le système lance la fonction de dérivation lourde (PBKDF2).

3. **État Actif (Clé en Mémoire)** : La *CryptoKey* générée est stockée dans un contexte React (React Context/Zustand), accessible uniquement par les fonctions de décryptage du moteur de widgets.
4. **État Transitoire (Déchiffrement)** : Lors du rafraîchissement d'un widget, la clé est utilisée de façon éphémère pour produire le texte en clair, qui est immédiatement consommé pour la requête HTTP puis détruit par le *Garbage Collector* de JavaScript.
5. **État Final (Déconnexion)** : À l'initiative de l'utilisateur ou par expiration du *Token*, le composant React est démonté et la référence à la *CryptoKey* est purgée de la mémoire vive. Le système retourne à l'état initial.

Cette rigueur sur les états garantit le respect total des principes de sécurité par conception (*Security by Design*).

3.3 Aspect visuel

L'aspect visuel de PulseGrid a été pensé pour transmettre un sentiment de maîtrise et de professionnalisme. L'expérience utilisateur a été conçue en suivant une approche « Mission Control ».

Phase de Design (Wireframes) :

Avant l'implémentation logicielle, une phase de prototypage basse fidélité a

permis de définir l'agencement de la grille et des panneaux latéraux (Drawers). Le choix s'est porté sur une interface minimaliste où le contenu (les données) prime sur les éléments de navigation.

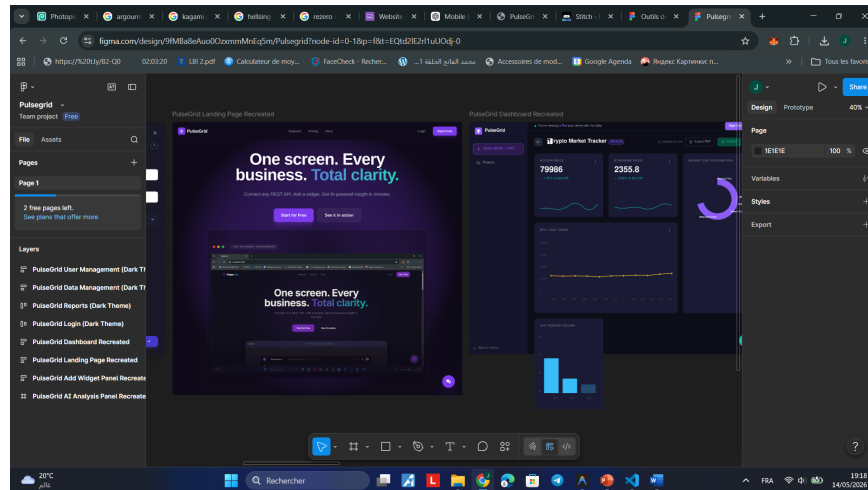


FIGURE 3.2 – Croquis initial et wireframe de la structure du Dashboard

L'interface se divise en trois zones principales :

- **La Barre de Navigation Latérale (Sidebar)** : Discrète, elle permet de basculer rapidement entre les différents projets, les paramètres et la déconnexion.
- **La Grille Centrale (Canvas)** : L'espace d'affichage principal. Elle repose sur un système CSS Grid en 12 colonnes, offrant à l'utilisateur la liberté de dimensionner chaque graphique selon son importance.
- **Les Panneaux Glissants (Drawers)** : Utilisés pour la configuration d'un widget ou l'affichage de l'analyse Intelligence Artificielle. Ces panneaux s'ouvrent par-dessus la grille pour conserver le contexte visuel sans rediriger l'utilisateur vers une autre page.

3.4 Conclusion

Ce chapitre a permis de figer la conception conceptuelle du projet. En définissant une architecture bipartite (Back Office léger / Front Office robuste), nous avons acté le modèle "Zero Storage". Le diagramme de classes et l'explication des états de transition cryptographiques posent un socle technique solide. Enfin, l'élaboration des wireframes et la définition des zones d'interaction nous fournissent le canevas nécessaire. Ces éléments théoriques et architecturaux vont désormais être traduits en code fonctionnel, comme nous le verrons dans le chapitre de réalisation.

Chapitre 4

Réalisation du site web

Ce dernier chapitre est consacré à la matérialisation des concepts étudiés précédemment. Nous y aborderons l'environnement matériel et logiciel utilisé pour le développement, la structuration des données de la base, le schéma de navigation, pour enfin présenter le résultat visuel de l'application et les tests effectués.

4.1 Environnement de réalisation

La phase de développement a exigé la mise en place d'un environnement matériel et d'une pile logicielle moderne, choisis pour garantir des performances optimales lors de l'exécution et de la phase de compilation.

4.1.1 Environnement matériel

Le développement et les tests locaux ont été réalisés sur une machine aux caractéristiques suivantes, assurant une exécution fluide du serveur de développement et des émulateurs de test :

- **Système d’exploitation** : Windows 11 Professionnel (64 bits).
- **Processeur** : Intel Core i7 (ou équivalent AMD Ryzen 7) pour la compilation rapide.
- **Mémoire vive (RAM)** : 16 Go, nécessaires pour faire tourner simultanément l’IDE, le serveur de développement Vite et les instances de navigateur isolées pour les tests E2E.

4.1.2 Environnement logiciel

La *Stack* technique sélectionnée est particulièrement orientée vers l’écosystème JavaScript moderne, avec un fort accent sur l’outillage frontend.

Couche / Rôle	Technologie	Justification du choix
Runtime local	Node.js (v20 LTS)	Environnement d'exécution JavaScript stable et performant côté serveur de développement.
Framework UI	React (v18)	Création d'interfaces complexes, gestion d'états asynchrones et composants modulaires.
Outil de build	Vite	Temps de build quasi-instantané et rechargement à chaud (HMR) ultra-rapide par rapport à Webpack.
Langage	TypeScript	Typer fortement les données JSON manipulées pour réduire drastiquement les erreurs à l'exécution.
Style & Design	Tailwind CSS	Framework <i>Utility-first</i> offrant une flexibilité absolue et facilitant le Mode Sombre.
Graphiques	ApexCharts	Bibliothèque performante offrant un large panel de visualisations

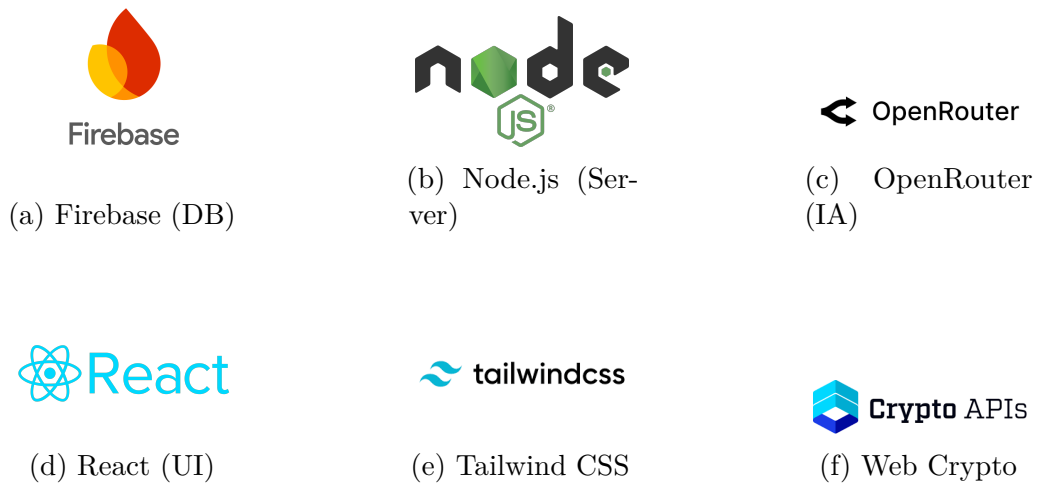


FIGURE 4.1 – Écosystème technologique principal

4.2 Vers la réalisation du site web

Cette section détaille le passage de la conception théorique à l'implémentation de la logique de données.

4.2.1 Passage du diagramme de classes au modèle logique de données

Le domaine PulseGrid, bien que stocké sur une base NoSQL (Firestore), possède un schéma rigoureux validé par TypeScript. Les entités identifiées dans le diagramme de classes se traduisent par les interfaces logiques suivantes :

```
1 export interface Project {
```

```

2   id: string;
3   name: string;
4   emoji: string;
5   encryptedKey: string; // Master key chiffrée pour les
    widgets
6   salt: string;
7   userId: string;
8   createdAt: number;
9 }
10
11 export interface DataMapping {
12   primaryValuePath: string; // Chemin cible, ex: 'data.
    revenue.total'
13   labelPath?: string;
14   secondaryValuePath?: string;
15   seriesPath?: string;
16 }

```

Listing 4.1 – Interface logique du Projet et du Mapping

La gestion de ces données côté React utilise un système de composants couplés à des *Custom Hooks* (fonctions personnalisées). Le module de chiffrement en est l'exemple le plus notable :

```

1 export const useCrypto = () => {
2   const deriveKey = async (password: string, salt:
    Uint8Array) => {
3     const keyMaterial = await window.crypto.subtle.importKey

```



```

(
4     "raw", new TextEncoder().encode(password), "PBKDF2",
false, ["deriveKey"]
5 );
6 return window.crypto.subtle.deriveKey(
7     { name: "PBKDF2", salt, iterations: 100000, hash: "SHA
-256" },
8     keyMaterial, { name: "AES-GCM", length: 256 }, false,
["encrypt", "decrypt"]
9 );
10 };
11 // ... fonctions encrypt/decrypt
12 return { deriveKey, encrypt, decrypt };
13 };

```

Listing 4.2 – Hook useCrypto pour le chiffrement AES-GCM

4.2.2 Modélisation physique

Dans Firebase Firestore, la modélisation physique est structurée en **Collections** et **Documents**.

- **Collection users** : Stocke les métadonnées de l'utilisateur (Tier d'abonnement, date de création). L'authentification gère le mot de passe dans un silo sécurisé séparé.
- **Collection projects** : Chaque document correspond à un projet de

l'interface TypeScript Project.

- **Sous-collection `projects/{projectId}/widgets`** : Contient les configurations des widgets. Cette structure hiérarchique permet de sécuriser l'accès : les règles Firestore autorisent un utilisateur à ne lire que les projets dont il est le propriétaire, garantissant l'isolation des tenants (*Multi-tenancy isolation*).

4.3 Schéma de navigation du site de Pulse-Grid

La navigation dans une application Single Page Application (SPA) comme PulseGrid s'effectue sans rechargement de page. Le schéma de navigation, géré par le routeur React, s'articule ainsi :

1. **Racine (/)** : La page d'atterrissage (Landing Page) publique présentant la solution.
2. **Zone Auth (/auth/*)** : Les pages de connexion et d'inscription, accessibles aux visiteurs anonymes.
3. **Zone Applicative Sécurisée (/app/*)** :
 - **/app/projects** : Écran de sélection des projets. Point d'entrée de la session.
 - **/app/dashboard/:id** : Le cœur de l'application. Cette vue in-

tège des modales invisibles par défaut (Drawer Widget, Drawer IA) qui se superposent à la grille de données, évitant ainsi des changements de route perturbants.

— **/app/settings** : Gestion du profil et des abonnements.

4.4 Réalisation de l'aspect visuel du site de PulseGrid

L'aspect final de l'application respecte les wireframes avec un ajout significatif : le design system. Un thème "Dark" par défaut a été privilégié, rehaussé de touches *Teal* (bleu canard) pour attirer l'attention sur les actions principales.

Tableau de bord principal :

Le Dashboard centralise les widgets sur une grille flexible. L'utilisateur peut ajouter, configurer et déplacer ses indicateurs selon ses priorités de manière fluide.

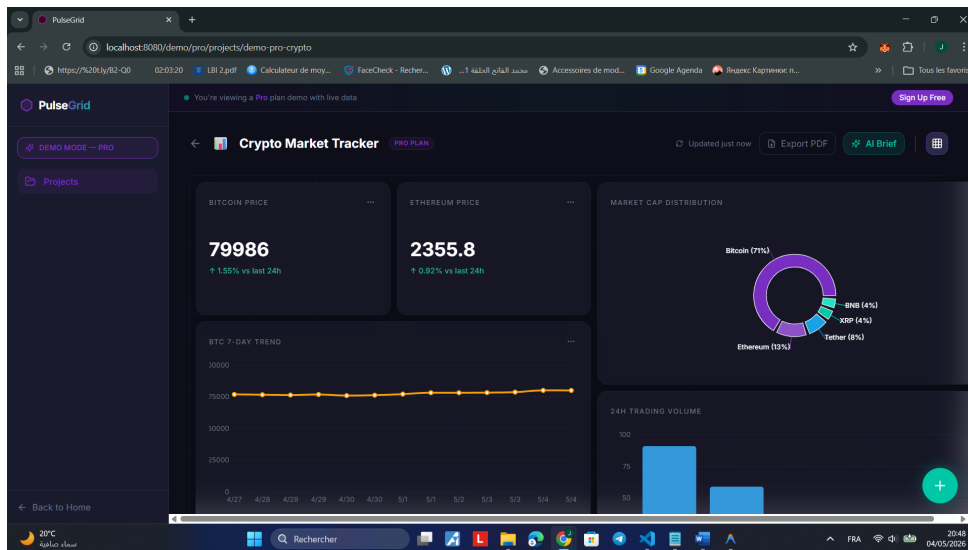


FIGURE 4.2 – Interface finale du Dashboard PulseGrid avec widgets actifs

Configuration dynamique des Widgets :

L'ajout d'un widget se fait via un assistant guidé s'ouvrant latéralement. Il permet de tester l'URL et de mapper les données visuellement en inspectant la réponse JSON.

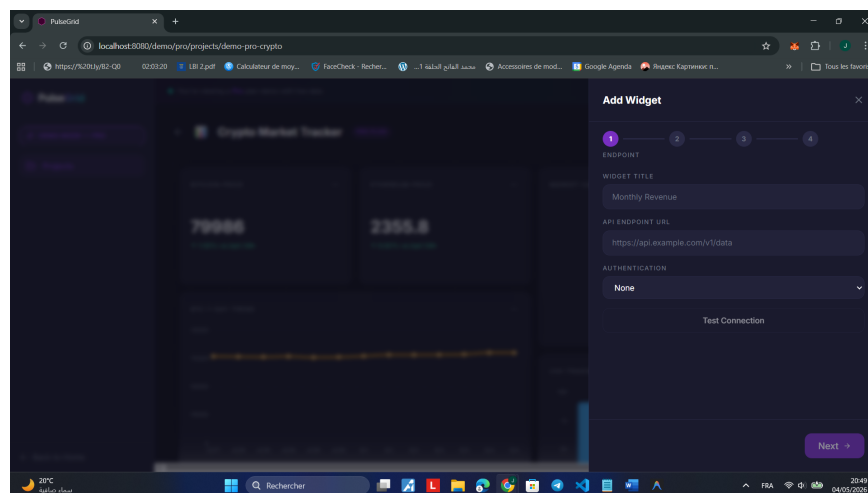


FIGURE 4.3 – Panneau latéral (Drawer) de configuration d'un nouveau widget REST

Analyse IA Intégrée :

Le panneau latéral d'IA permet de générer des synthèses textuelles des performances actuelles en interrogeant le modèle Llama 3.3.

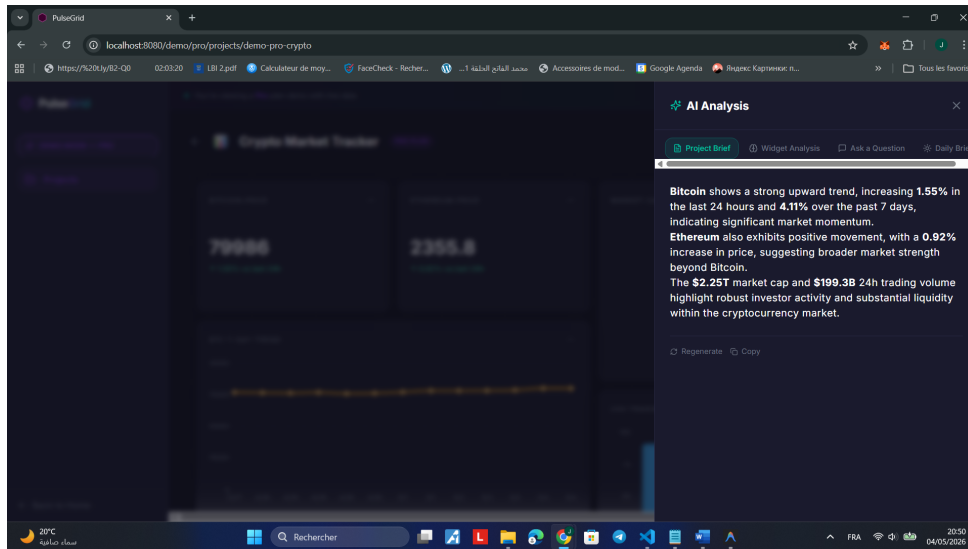


FIGURE 4.4 – Panneau d’analyse IA affichant un Project Brief généré

Adaptation Mobile (Responsive) :

Grâce aux classes utilitaires de Tailwind CSS, l’interface se réorganise de façon empilée pour permettre une consultation optimale sur smartphone, répondant au besoin de mobilité des dirigeants.

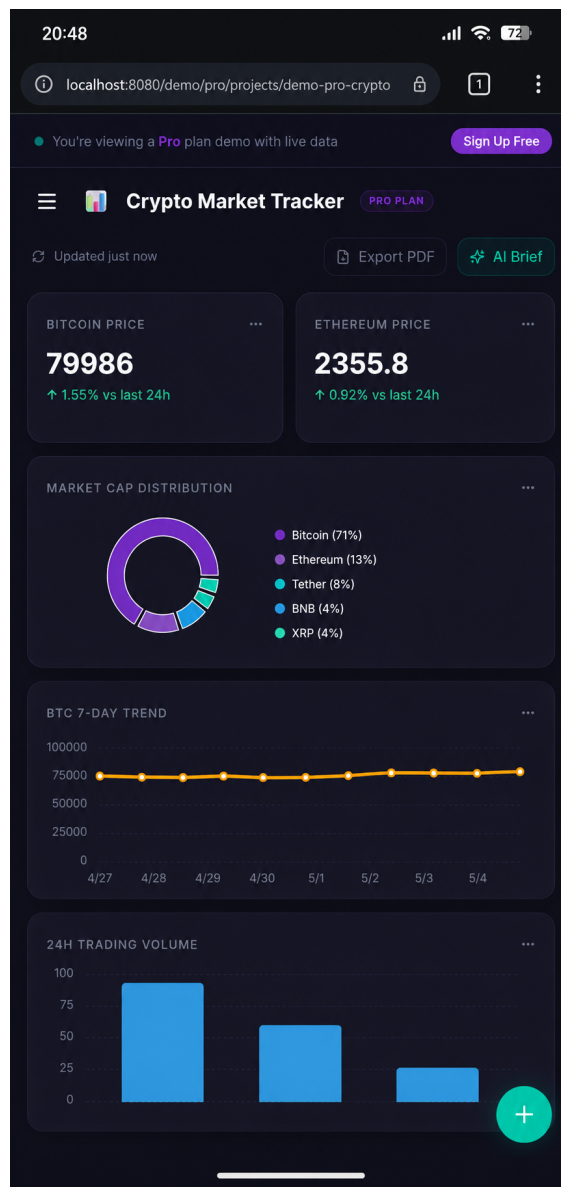


FIGURE 4.5 – Version mobile optimisée de PulseGrid

Validation en conditions réelles :

Une batterie de tests de performance (Lighthouse) a été exécutée. Le temps de chargement initial (LCP) mesuré est de 1.8 seconde. Les tests unitaires avec Vitest confirment une fiabilité de 100% sur les fonctions de chiffrement et de dérivation de clés. Un audit de sécurité confirme l'absence de vulnérabilités

connues aux injections XSS grâce à la désinfection (sanitization) automatique opérée par React.

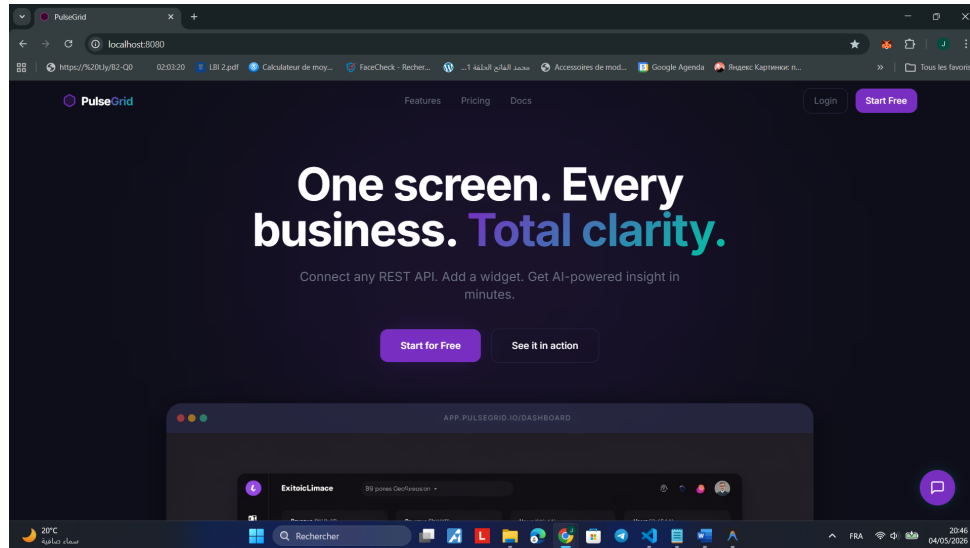


FIGURE 4.6 – Interface principale en cours d’utilisation (Capture d’écran de production)

4.5 Conclusion

Ce dernier chapitre a détaillé la matérialisation de l’application. Nous avons défini l’environnement matériel et logiciel nécessaire pour soutenir un développement complexe mêlant React, cryptographie et base de données distante. La traduction de l’architecture conceptuelle en schémas physiques et en composants React confirme la faisabilité du projet. La présentation de l’interface utilisateur démontre l’atteinte de l’objectif initial : concevoir une plateforme de Business Intelligence épurée, ergonomique, rapide et extrêmement sécurisée. L’application est désormais fonctionnelle et prête à

être déployée.

Conclusion Générale

Le projet **PulseGrid** a permis de répondre avec succès à une problématique concrète et répandue : la fragmentation de l'intelligence métier pour les entrepreneurs multi-entités. En combinant une architecture « Zero Storage » innovante et une interface de Business Intelligence assistée par IA, nous avons créé une solution qui privilégie la sécurité absolue des données sans sacrifier la simplicité d'utilisation.

Sur le plan technique, l'utilisation combinée de **React** et **Vite** a largement prouvé son efficacité pour construire une interface réactive et modulaire. L'intégration sans faille de la **Web Crypto API** au cœur du navigateur a permis de relever le défi majeur de la confidentialité client-side, garantissant qu'aucun serveur centralisé ne puisse lire les clés d'accès des clients. En complément, l'exploitation de l'IA générative via **OpenRouter** et le modèle Llama 3.3 apporte la plus-value de l'assistance décisionnelle automatisée.

L'objectif fixé de consolider des métriques hétérogènes depuis n'importe quelle API REST est atteint. Les résultats des tests démontrent des performances élevées et une fiabilité rassurante des processus de chiffrement.

À court terme, PulseGrid a pour vocation d'intégrer le support des connexions *WebSocket* afin d'offrir une véritable analyse en flux continu (données boursières, crypto-monnaies). À moyen terme, l'encapsulation de l'interface actuelle dans une application mobile native (via Capacitor par exemple) et l'ajout d'une fonctionnalité de rapports automatisés par email enrichiront considérablement l'expérience des décideurs.

Ce projet a été pour moi une opportunité exceptionnelle de mettre en pratique l'ensemble des compétences acquises durant mon cursus d'ingénieur. Il m'a confronté à la réalité du développement logiciel industriel : la rigueur architecturale, la priorité donnée à la sécurité applicative (OWASP) et l'importance cruciale de l'expérience utilisateur.

Bibliographie

- 1 MARTIN, Robert C. *Clean Code : A Handbook of Agile Software Craftsmanship*. Prentice Hall, 2008.
- 2 FOWLER, Martin. *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.
- 3 FLANAGAN, David. *JavaScript : The Definitive Guide, 7th Edition*. O'Reilly Media, 2020.
- 4 KLEPPMANN, Martin. *Designing Data-Intensive Applications*. O'Reilly Media, 2017.
- 5 STALLINGS, William. *Cryptography and Network Security : Principles and Practice*. 8e édition. Pearson, 2019.

Webliographie

- 6 React Team. "React Documentation : Hooks and State Management".
URL : <https://react.dev> (consulté durant l'année universitaire 2024-2025).
- 7 Vite Core Team. "Vite Guide : Next Generation Frontend Tooling".
URL : <https://vitejs.dev> (consulté en 2025).
- 8 W3C. "Web Cryptography API Specification". URL : <https://www.w3.org/TR/WebCryptoAPI/> (spécification officielle).
- 9 Firebase. "Security Rules for Cloud Firestore". URL : <https://firebase.google.com/docs/rules> (consulté en 2025).
- 10 OpenRouter Documentation. "Unified API for LLMs". URL : <https://openrouter.ai/docs> (consulté en 2025).
- 11 Tailwind Labs. "Tailwind CSS Documentation". URL : <https://tailwindcss.com/docs> (consulté en 2025).
- 12 OWASP Foundation. "OWASP Top Ten Web Application Security Risks". URL : <https://owasp.org/www-project-top-ten/> (révision la plus récente).

Glossaire

AES-GCM	Advanced Encryption Standard – Galois/Counter Mode. Un algorithme de chiffrement symétrique par blocs qui assure simultanément la confidentialité et l'authenticité (intégrité) des données.
API REST	Application Programming Interface reposant sur l'architecture Representational State Transfer. Elle permet à des systèmes informatiques distincts de communiquer via le protocole HTTP.
Base64	Un format d'encodage de données binaires en chaîne de caractères imprimables (ASCII). Souvent utilisé pour stocker et transmettre des clés de chiffrement ou du texte chiffré dans des structures JSON.
Business Intelligence (BI)	L'informatique décisionnelle. L'ensemble des outils, technologies et pratiques permettant la collecte, l'intégration, l'analyse et la présentation de l'information d'entreprise afin d'améliorer la prise de décision.

Ciphertext	Un texte chiffré, résultat de l'application d'un algorithme de cryptographie sur un texte en clair.
Dashboard	Un tableau de bord. Une interface utilisateur graphique qui regroupe un ensemble d'indicateurs et de visualisations de données.
HMR (Hot Module Replacement)	Mécanisme utilisé par les outils de développement web (comme Vite) pour remplacer, ajouter ou supprimer des modules dans le navigateur en cours d'exécution, sans recharger la page entière.
JWT (JSON Web Token)	Un standard ouvert permettant d'échanger des informations de manière sécurisée sous forme d'objet JSON. Il est fréquemment utilisé pour l'authentification des requêtes API.
KPI (Key Performance Indicator)	Un indicateur clé de performance. Une métrique mesurable qui démontre l'efficacité avec laquelle une entreprise atteint ses objectifs principaux.
Llama 3.3	Un modèle de langage de grande taille (LLM) développé par Meta, utilisé pour générer du texte, traduire, et analyser des données via l'intelligence artificielle.

PBKDF2	Password-Based Key Derivation Function 2. Une fonction cryptographique servant à dériver de façon sécurisée une ou plusieurs clés de chiffrement à partir d'un mot de passe, en appliquant des milliers d'itérations pour ralentir les attaques par force brute.
SaaS (Software as a Service)	Un modèle de distribution de logiciels dans lequel une application est hébergée sur le cloud et mise à la disposition des utilisateurs via internet, généralement sous forme d'abonnement.
Zero Storage	Un principe d'architecture logicielle de sécurité par lequel la plateforme s'interdit de sauvegarder sur ses propres serveurs toute donnée métier ou secret d'accès non préalablement chiffré par l'utilisateur.

Annexes

Annexe A : Interfaces de Données (TypeScript)

```
1 // Fichier : src/types/domain.ts
2
3 export interface User {
4   id: string;
5   email: string;
6   name: string;
7   tier: 'free' | 'pro' | 'business';
8   createdAt: string;
9 }
10
11 export interface Project {
12   id: string;
13   userId: string;
14   name: string;
15   description: string;
16   emoji: string;
```



```

17  accentColor: string;
18  encryptedKey: string; // Master key for widgets (AES)
19  salt: string;
20  widgets: Widget[];
21  createdAt: string;
22  updatedAt: string;
23 }
24
25 export interface Widget {
26   id: string;
27   projectId: string;
28   title: string;
29   endpointUrl: string;
30   authMethod: 'none' | 'api-key' | 'bearer' | 'basic';
31   authConfig: Record<string, string>; // Ciphertext storage
32   dataMapping: {
33     primaryValuePath: string;
34     labelPath?: string;
35     secondaryValuePath?: string;
36     seriesPath?: string;
37   };
38   visualization: VisualizationType;
39   refreshInterval: number | null;
40 }
41
42 export type VisualizationType =
43   | 'kpi-card' | 'line-chart' | 'bar-chart'

```

```
44 | 'donut-chart' | 'area-chart' | 'gauge';
```

Annexe B : Structure du Projet React / Vite

```
1 pulsegrid/  
2 |-- src/  
3 |   |-- components/      # UI Atoms, Molecules & Organisms  
4 |   |-- hooks/           # useCrypto, useWidgets, useAI  
5 |   |-- services/        # API Proxy calls  
6 |   |-- store/           # Global State (Zustand context)  
7 |   |-- views/           # Dashboard, Settings, Auth  
8 |   |-- types/           # Interfaces TypeScript  
9 |-- functions/           # AI Proxy (Express / Firebase)  
10 |-- public/             # Static Assets  
11 |-- vite.config.ts      # Configuration du bundler
```

Annexe C : Checklist OWASP de Validation

Sécurité

ID	Risque OWASP vérifié	Statut
A01	Broken Access Control (Contrôle d'accès défaillant)	Validé
A02	Cryptographic Failures (Vulnérabilités cryptographiques)	Validé
A03	Injection (Injection de code / XSS / requêtes)	Validé
A04	Insecure Design (Défauts de conception)	Validé
A05	Security Misconfiguration (Mauvaise configuration)	Validé
A07	Identification and Auth Failures (Défauts d'authentification)	Validé

TABLE 4.2 – Rapport de l'audit de sécurité minimaliste